

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_e1b943dd-dac\agent-ad175df51d8d76e1d.jsonl`  
- SHA-256: `b660168b106a728f9bdc0f23a7c43a6ae8068bd907b14eaa643ddec0e26a7d34`  
- Source modified: `2026-07-06T19:19:42+00:00`  
- Imported at: `2026-07-08T16:00:12+00:00`  
- project: `wf_e1b943dd-dac`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T19:18:54.444Z)

Reviewing GitHub PR #40 of the repo at C:/Users/avidu/Projects/Telegram ? a DRAFT, DOCS-ONLY PR (branch docs/track-f-policy-pipeline).
It adds ONE file: docs/adr/0012-policy-pipeline.md (ADR 0012, status Proposed) ? the Track F design doc for a four-tier "policy pipeline" (Tier 1 metadata gates / Tier 2 topic preferences / Tier 3 content safety / Tier 4 deep inspection), a PolicyStage protocol + StageResult explanation trace, and 7 phased follow-up issues F1-F7.
No code changes. Read the ADR:
git -C C:/Users/avidu/Projects/Telegram show docs/track-f-policy-pipeline:docs/adr/0012-policy-pipeline.md

This is a DESIGN review, not a code review ? there is no gate to run. Judge the ADR as a design contract:
soundness, completeness, internal consistency, accuracy of its claims, and consistency with the codebase and the other ADRs (docs/adr/0001..0011 exist on main; read any you need via 'git -C C:/Users/avidu/Projects/Telegram show main:docs/adr/<file>').
Relevant current code (on main): src/tg_video_filter/models.py (FilterConfig, ContentItem, Source, Policy),
src/tg_video_filter/db.py (load_filter_config/save_filter_config shims, sources.watched gate should_ingest_source, delivery_attempts/record_delivery), src/tg_video_filter/telethon_client.py (the live handler: watched gate -> dedup -> FilterEngine.evaluate -> insert -> deliver). Track E (sources.watched opt-in gate) and Track C (delivery_attempts) just merged.

Ground rules:

- READ actual files, cite ADR section / file:line. You MAY run C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe to check claims (e.g. does FilterConfig ignore/drop unknown JSON keys? does ContentItem carry caption text? is content_item.text populated?).
- This is a DRAFT asking for design feedback ? frame findings as design gaps/risks/inaccuracies, not merge-blockers.
 - Severity: high (a claim that is false, or a design gap that would break an F-phase / cause data loss), medium (unspecified interaction, inconsistency, missing dependency), low (wording, invalid example, minor).
- Report only issues in THIS ADR. No praise. Be concrete: what's wrong, why it matters for implementation.

Concept	Current state
---	---
`FilterConfig` / `Policy`	One JSON blob per account. Flat duration/resolution/size/exclude rules. AND semantics.

```
| `FilterEngine` | Evaluates all rules; returns `FilterResult(passed, reasons)`. |
| `policies` table | One row per account (is_default=1). `config_json` is the `FilterConfig` blob. |
| Security / topic / deep-inspect | Nothing. Codec/keyword/MIME filters and ffmpeg were deferred to v2 in the original plan. |
```

What we want

A **policy** should be a pipeline of ordered **stages**, each with a well-defined cost, inputs, and explainable output. The pipeline runs **after** dedup and **before** delivery. Users should be able to express topic preferences ("I like AI and philosophy videos"), security rules ("reject links from sketchy domains"), and view **why** a specific item was accepted or rejected.

Decision

Stage model ? four ordered tiers

The pipeline is split into four tiers, ordered by cost (cheapest first). Later stages only run if all earlier stages pass:

```
...
????????????????????????????????????????????????????????????
?                Policy Pipeline                                ?
?                                                         ?
? Tier 1 ? Metadata gates  (always runs, zero I/O)           ?
?   duration, resolution, size, codec, mime, GIF/round       ?
?   ? extends today's FilterEngine                          ?
?                                                         ?
? Tier 2 ? Topic preferences (opt-in, metadata-only)         ?
?   source allowlist, keyword match, channel topic tags      ?
?   ? user-expressed interests filter the stream             ?
?                                                         ?
? Tier 3 ? Content safety  (opt-in, metadata + light I/O)    ?
?   URL/domain reputation, file metadata sanity,             ?
?   caption text scanning                                    ?
?   ? cheap checks; no download                              ?
?                                                         ?
? Tier 4 ? Deep inspection (opt-in, heavy I/O)               ?
?   ffmpeg, perceptual hash, downloaded content scan         ?
?   ? gated behind explicit user opt-in + budget             ?
????????????????????????????????????????????????????????????
...
```

Stage interface ? `PolicyStage`

Each stage is a pure function (or async function for I/O stages) that takes a `PolicyContext` and returns a `StageResult`:

```
```python
@dataclass
class PolicyContext:
 """Immutable snapshot of what the stage can inspect."""
 content_item: ContentItem # from ADR 0011
 source: Source # the source it came from
 policy: Policy # the owning policy config

@dataclass
class StageResult:
 passed: bool
 reason: str # human-readable explanation
 stage_name: str # e.g. "duration-gate", "url-reputation"
 metadata: dict[str, object] = field(default_factory=dict) # for explainability UI
```

```
class PolicyStage(Protocol):
 """A single gate in the pipeline. Stateless; may be async for I/O stages."""
 async def evaluate(self, ctx: PolicyContext) -> StageResult: ...
 ...
```

The pipeline evaluates stages in order and **short-circuits** on the first failure (don't waste I/O on an already-rejected item). The accumulated `StageResult`` list is the **explanation trace** ? every decision is auditable.

## Topic preferences ? Tier 2

Topic preferences are stored as a JSON array in `policies.config_json`` under a `topics`` key:

```
```json
{
  "min_duration_s": 5,
  "topics": {
    "include": ["AI", "Philosophy", "Soccer"],
    "exclude": ["Politics", "Crypto"],
    "mode": "any" // "any" = match any include topic; "all" = match all
  }
}
```
```

Topic matching is **metadata-only** in v1: source title substring match, message caption keyword match, and (future) channel topic tags from Telegram's channel metadata. No ML classification in the design ? that's a separate issue.

## Security stages ? Tier 3

Security stages are **opt-in** and configured per-policy. The design defines three initial stages (implementation deferred to separate issues):

1. **Source allowlist**: reject items from sources not in the `allowlist`` (an explicit list of trusted `provider_source_ids`).
2. **URL/domain reputation**: check links in captions against a configurable blocklist. Metadata-only ? no page fetch.
3. **Duplicate suppression**: enhanced dedup that compares by `content_fingerprint`` across accounts (already handled by `content_items.dedup_key``; this stage adds cross-policy duplicate detection within an account's delivery streams).

Each security stage records its decision in `StageResult.metadata`` for explainability (e.g. `{"blocked_domain": "sketchy-site.com"}``).

## Deep inspection ? Tier 4

Deep inspection (ffprobe, perceptual hash, downloaded content scan) is **gated** behind explicit opt-in because it violates ADR 0003's no-download principle. The opt-in is per-policy (`deep_inspect: true`` in `config_json``) and the stage has a **budget** (max bytes per item, max concurrent downloads) enforced by the policy engine.

This tier is explicitly deferred to a follow-up issue ? the design only **reserves the slot** in the pipeline model so it doesn't require a schema migration when implemented.

## Policy evaluation flow (post-dedup, pre-delivery)

```
```python
async def evaluate_pipeline(
```

```

    ctx: PolicyContext,
    stages: list[PolicyStage],
) -> tuple[bool, list[StageResult]]:
    results: list[StageResult] = []
    for stage in stages:
        result = await stage.evaluate(ctx)
        results.append(result)
        if not result.passed:
            return False, results # short-circuit
    return True, results
...

```

The caller (telethon_client or future Bot API handler) records the results via `delivery\_attempts.last\_error` (for failed items) or a new `policy\_decisions` table (for explainability ? deferred to implementation).

Schema ? minimal changes

****No schema migration**** is required for this design. The `policies` table already holds a `config\_json` JSON blob. The pipeline is a code concept:

- `policies.config\_json` gains optional `topics`, `allowlist`, `security\_checks`, `deep\_inspect` keys ? all backwards-compatible with the existing `FilterConfig` Pydantic model (extra fields are ignored).
- Future `policy\_decisions` table (deferred) would store the explanation trace for debugging: `(content\_item\_id, policy\_id, stage\_name, passed, reason, metadata\_json, evaluated\_at)`.

Implementation phasing (future issues)

Phase	Scope	Dependencies
---	---	---
F1	Policy engine prototype	`PolicyStage` protocol, `evaluate_pipeline()`, wire into live-message handler None
F2	Topic preferences	`topics` config parsing, source title/caption keyword matching F1
F3	Source allowlist	`allowlist` config, skip non-allowlisted sources F1
F4	URL/domain reputation	Domain blocklist, caption link extraction F1
F5	Deep inspection	ffprobe/perceptual hash, download budget enforcement F1, ADR 0003 waiver
F6	Policy decisions table	Audit trail storage, explainability UI F1
F7	Explainability endpoint	`/why <content_item_id>` command showing the explanation trace F6

Each phase is a separate, mergeable PR ? none are bundled here. This ADR only defines the ****design contract**** the phases code against.

Consequences

****Positive****

- Unifies topic, security, and quality gates into a single explainable framework ? no ad-hoc if-chains scattered across handlers.
- Cheap gates always run first; expensive enrichment is only reached for items that pass the cheap gates.
- `StageResult` list provides an audit trail (explainability) without additional DB lookups at decision time.
- Backwards-compatible: `policies.config\_json` already exists; adding optional keys requires no migration.
- Each implementation phase is independently shippable behind feature flags.

****Negative****

- Four tiers add conceptual complexity vs. today's flat filter.
- `PolicyStage` protocol requires all stages to conform to the same

- interface, which may be awkward for deeply different checks (metadata-only vs. I/O-heavy). Mitigated by the `metadata` dict catch-all.
- Topic matching is string-based substring matching in Phase 1 ? not semantic. A future ML stage would replace it, but it's good enough for a v1.
- Deep inspection violates ADR 0003 and must be explicitly opt-in with a budget to avoid runaway costs.

Alternatives considered

- ****Do nothing; keep flat FilterConfig.**** Rejected ? the umbrella exists to dissolve the conflation of quality, topic, and security into a single boolean. A pipeline is the natural generalization.
- ****Inline each stage in the handler**** (no protocol). Rejected ? makes it impossible to test stages independently, reorder them, or add new ones without touching the hot path.
- ****Use a rules engine / DSL.**** Rejected ? over-engineered for <100 users. A Python list of callables is simpler, testable, and introspectable.
- ****Bundle all phases into a single PR.**** Rejected ? scope discipline. Each phase is independently shippable; this ADR defines the shared interface they code against.

Open questions for the decision owner

1. ****`StageResult.metadata` shape**** ? dict or typed per-stage? Recommend: dict for Phase 1 (flexible), typed in Phase 2 (once the shape stabilizes).
2. ****Policy decisions table**** ? store every decision, or only failures? Recommend: every decision (explainability), with a TTL-based cleanup for old traces.
3. ****Topic matching**** ? substring only, or regex/glob? Recommend: substring (case-insensitive) for v1; natural-language matching in a follow-up.
4. ****Deep inspection budget**** ? per-item byte limit or per-account bandwidth cap? Recommend: per-item limit (10 MB default) with a per-account monthly cap enforced at the app level.

References

- ADR 0003 ? metadata-first probing (the no-download principle that Tier 4 must opt in to violate).
- ADR 0009 ? selective channel scanning (the `topics.allowlist` concept generalizes `sources.watched`).
- ADR 0011 ? provider-neutral domain schema (the `policies` table this design extends, `ContentItem` as the `PolicyContext` input).
- Issue #31 ? Track F (this ADR's tracking issue).
- Issue #25 ? umbrella; this is the final track.

4. assistant (2026-07-06T19:19:03.018Z)

Now let me check the specific claims. Let me look at ADR 0009 and verify the three cited "homes" for the allowlist concept.

5. user (2026-07-06T19:19:04.936Z)

ADR 0009: Selective channel scanning and multiplexed routing

- ****Status****: Accepted
- ****Date****: 2026-07-04
- ****Decision owner****: project owner

Context

Today the userbot scans **every** channel/group the account is a member of. ``manager.py`` registers ``events.NewMessage(incoming=True)`` with no ``chats=`` filter, and ``on_new_message`` processes every video regardless of source. The ``channels`` table exists but is populated lazily as a *record* of seen channels; it is not used as a filter.

The product vision is a **multiplexing video re-distribution system**:

- A small set of high-quality **source** channels (e.g. spiritual, classical music) are scanned for incoming videos.
- Matches are routed to themed **destination** broadcast channels (e.g. a Reggae channel, a Rap channel) that subscribers opt into.
- A user therefore subscribes to *outputs* by topic, not to *inputs* by source.

This requires two capabilities the current architecture does not have:

1. **Selective scanning** ? choose which source channels are watched, rather than ingesting all of them.
2. **Per-source ? per-destination routing** ? a match from source channel X is delivered to a specific destination channel Y, instead of the single global ``delivery_target`` (ADR 0005).

Decision

Add three additive, independently-shippable capabilities:

1. Selective scanning (watched-flag)

Add a ``watched`` flag to the ``channels`` table. The default is locked as **opt-in: `DEFAULT 0`**, so a freshly-onboarded user scans nothing until they explicitly ``/watch`` a channel. This matches the multiplexing product shape (sources are a *curated* set) and avoids blindly re-ingesting every chat the account is a member of.

Gate ``on_new_message`` at the top: after computing ``channel_id``, look up the row and ``return`` early if ``watched`` is false, before any dedup or filter work.

This is a small migration plus ~5 lines in ``telethon_client.py``.

Discoverability note. With ``DEFAULT 0`` plus the current lazy population (a ``channels`` row is only inserted when a message is first seen), opt-in is necessary but not sufficient for discoverability: a channel the account has never received a message from has no row to flag. The ``/channels`` command (step 2 below) therefore upserts candidate rows from ``client.get_dialogs()`` so users can browse and watch channels they have not yet received traffic from. The flag default and the enumeration flow are orthogonal ? both are required.

2. Channel enumeration and management commands

New Saved Messages commands (and, via ADR 0008, bot commands) building on the existing ``commands.py`` pattern:

- ``/channels`` ? list the user's channels (via ``client.get_dialogs()``, already used in ``backfill.py``) with their watched state.
- ``/watch <#name|id>`` ? set ``watched=1`` for a source channel.
- ``/unwatch <#name|id>`` ? set ``watched=0``.
- ``/watchlist`` ? list only watched sources.

3. Per-source ? per-destination routing rules

Introduce a ``routing_rules`` table (the canonical name; issue #25's ``routes``

refers to the same concept and should be aligned to `routing\_rules`), e.g.:

```
...
routing_rules (
    user_id            INTEGER NOT NULL REFERENCES users(id),
    source_channel_id  INTEGER NOT NULL,
    dest_channel_id    INTEGER NOT NULL,    -- a broadcast channel / @username
    filter_config_id   INTEGER,            -- optional per-route filter override
    PRIMARY KEY (user_id, source_channel_id, dest_channel_id)
)
```

6. assistant (2026-07-06T19:19:09.489Z)

ADR 0009 does not itself use the term "topics.allowlist" ? that's the ADR 0012 author's characterization. Now let me pin down the three placements the finding cites, and check the FilterConfig extra-fields claim.

7. user (2026-07-06T19:19:10.954Z)

No matches found

8. assistant (2026-07-06T19:19:17.253Z)

The file is only in the branch, not on disk. The finding's three placements are all confirmed from the ADR text I already read:

1. ****Tier 2 ASCII box**** (Decision, four-tier diagram): "source allowlist, keyword match, channel topic tags"
2. ****Tier 3 prose**** (Security stages): "1. ****Source allowlist****: reject items from sources not in the `allowlist`"
3. ****Schema section****: "config_json gains optional `topics`, `allowlist`, `security\_checks`, `deep\_inspect` keys" ? `allowlist` as a top-level sibling of `topics`"
4. ****References****: "ADR 0009 ? ... the `topics.allowlist` concept generalizes `sources.watched`"

Let me verify the FilterConfig extra-fields claim and the sources.watched gate to assess the "belongs nowhere" compounding argument.

9. user (2026-07-06T19:19:19.572Z)

```
src/tg_video_filter/models.py:44:     model_config = ConfigDict(frozen=True)
src/tg_video_filter/models.py:65:     model_config = ConfigDict(frozen=True)
src/tg_video_filter/models.py:88:     model_config = ConfigDict(frozen=True)
src/tg_video_filter/models.py:104:    model_config = ConfigDict(frozen=True)
src/tg_video_filter/models.py:131:    model_config = ConfigDict(frozen=True)
src/tg_video_filter/models.py:159:    model_config = ConfigDict(frozen=True)
src/tg_video_filter/models.py:196:    model_config = ConfigDict(frozen=True)
src/tg_video_filter/models.py:230:    model_config = ConfigDict(frozen=True)
src/tg_video_filter/models.py:253:    model_config = ConfigDict(frozen=True)
src/tg_video_filter/models.py:285:    model_config = ConfigDict(frozen=True)
---watched gate---
src/tg_video_filter/db.py:99:     watched            INTEGER NOT NULL DEFAULT 0,
src/tg_video_filter/db.py:522:     # 1a. Backfill sources from channels first (known title +
watched=1). Order
src/tg_video_filter/db.py:523:     #     matters: channels-first so title/watched data wins; the
videos-based
src/tg_video_filter/db.py:528:             (account_id, provider, provider_source_id, title,
kind, watched,
src/tg_video_filter/db.py:539:     #     ? the channels-based row above survives with its title +
watched=1.
src/tg_video_filter/db.py:543:             (account_id, provider, provider_source_id, kind,
watched, seen_first_at)
src/tg_video_filter/db.py:851:             watched=bool(row["watched"]),
src/tg_video_filter/db.py:1110:     ``watched`` is **user-controlled state** (toggled only via
src/tg_video_filter/db.py:1111:     :func:`set_source_watched` / the ``/watch`` and ``/unwatch``
```


commands) and is
src/tg_video_filter/db.py:1114: so clobbering ``watched`` here would silently un-watch a
channel the user

10. assistant (2026-07-06T19:19:32.029Z)

The finding is fully verifiable. Let me confirm the specific facts:

- The FilterConfig model uses `ConfigDict(frozen=True)` but no `extra="ignore"` set explicitly. Pydantic v2 default is `extra="ignore"`, so the ADR's claim that extra fields are ignored is actually correct (not part of this finding). This is relevant to the "compounds Finding 1" aside but not the core placement inconsistency.
- The `watched` gate exists (`sources.watched DEFAULT 0`, `set\_source\_watched`, `/watch`/`/unwatch`), confirming that source-level allowlisting is already realized by Track E ? supporting the finding's compounding argument.

Now assessing the core finding: is the allowlist genuinely placed in three inconsistent homes?

****Verified placements:****

1. Tier 2 ASCII box lists "source allowlist" as a topic-preference example.
2. Tier 3 prose defines "Source allowlist" as security stage #1, reading from an `allowlist` key.
3. Schema section lists `allowlist` as a top-level `config\_json` key (sibling of `topics`), NOT nested under `topics`.
4. References calls it `topics.allowlist` (nested under topics).

So the config key location is genuinely ambiguous: `topics.allowlist` (References) vs. top-level `allowlist` (Schema) vs. an unnamed key backing the Tier-3 stage. And the tier ownership is genuinely split: the same "source allowlist" appears both in Tier 2 (topic preferences) and Tier 3 (security stages). F3 ("Source allowlist") is filed as a security-tier phase, yet Tier 2's box also claims it.

One partial nuance on the finding's framing: the finding says the config example "nests allowlist as a Tier-3 security_checks concept," but the Schema section actually lists `allowlist` and `security\_checks` as *separate* sibling keys ? so it's not nested inside `security\_checks`. That makes the inconsistency arguably worse (four distinct signals, not three), not weaker. The finding's core claim holds; only that one sub-detail of the evidence is slightly imprecise.

The finding is real, internal to this ADR, and would affect the config schema and F3 ordering. Not a false claim or data-loss issue, so medium severity is correct for a draft design doc.

11. user (2026-07-06T19:19:42.153Z)

Structured output provided successfully